

Mini Project 01 — Titanic Survival Classification

Technical report: from-scratch (NumPy) versus scikit-learn implementations. Seed 42 throughout; one stratified 80/20 split (712 train / 179 test) shared by every model.

1. Objective and minimal preprocessing

Objective: predict whether a passenger survived ($Survived = 1$) from class, sex, age, family size, fare and port of embarkation. Four identifier or free-text columns were dropped (PassengerId, Name, Ticket, Cabin, the latter 77% missing). The data were split first, stratified on the target (38.3% survivors in train, 38.6% in test), and every statistic was then learned from the training rows only to avoid leakage: median imputation for Age (28.5) and Fare, mode imputation for Embarked, a binary Sex column, one-hot Pclass and Embarked with a reference level dropped, and $\log(1+Fare)$ followed by standardisation of the four numeric columns. The result is nine numeric features. Hyper-parameters were chosen by five-fold stratified cross-validation (CV) on the training set using ROC-AUC; the test set was used once per model.

2. Key results

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Avg. prec.	Log-loss
[B] LogReg, no penalty	0.799	0.780	0.667	0.719	0.838	0.788	0.467
[B] LogReg, L1	0.816	0.810	0.681	0.740	0.844	0.795	0.464
[B] LogReg, L2	0.821	0.836	0.667	0.742	0.838	0.797	0.476
[B] Bagging of trees	0.816	0.810	0.681	0.740	0.846	0.826	0.460
[B] AdaBoost, stumps	0.799	0.770	0.681	0.723	0.795	0.764	0.548
[C] LogReg, no penalty	0.799	0.780	0.667	0.719	0.838	0.788	0.467
[C] LogReg, L1	0.816	0.810	0.681	0.740	0.844	0.795	0.464
[C] LogReg, L2 (own CV)	0.799	0.780	0.667	0.719	0.838	0.789	0.467
[C] LogReg, L2 (same lambda as B)	0.821	0.836	0.667	0.742	0.838	0.797	0.476
[C] Random forest	0.816	0.821	0.667	0.736	0.838	0.820	0.467
[C] Bagging of trees	0.804	0.870	0.580	0.696	0.836	0.828	0.448
[C] AdaBoost, stumps	0.799	0.770	0.681	0.723	0.826	0.772	0.603
[C] Gradient boosting	0.782	0.788	0.594	0.678	0.799	0.770	0.520
[C] Hist. gradient boosting	0.799	0.800	0.638	0.710	0.810	0.783	0.511

Table 1. Test-set metrics (threshold 0.5). [B] = NumPy from scratch (shaded), [C] = scikit-learn. One passenger equals 0.56 accuracy points; 95% bootstrap intervals are about ± 0.06 for accuracy and ± 0.065 for AUC.

3. Logistic regression variants

The from-scratch models minimise mean log-loss plus a penalty of $\lambda/2$ times the squared L2 norm of the weights (L2) or λ times their L1 norm (L1), using gradient descent with the exact step size $1/L$; L1 uses a proximal soft-thresholding step. The analytic gradient matched a numerical gradient to $1e-10$, and with the same λ the coefficients agree with scikit-learn to better than $3e-7$, with identical test labels (rows 1–3 of Table 1 versus rows 6, 7 and 9). This supports the mathematical accuracy of Section B.

Regularisation changed discrimination very little: CV-AUC was 0.852–0.855 for any λ up to 0.01, then fell (L2 reached 0.826 at $\lambda = 0.3$, while L1 removed every feature and collapsed to 0.5). With 712 rows and only nine features there is little variance to remove, so the unregularised model is already close to optimal; test AUC was 0.838, 0.844 and 0.838 for none, L1 and L2. Its main effect is on the coefficients. L2 (at $\lambda = 0.01$) shrank the weight norm from 3.35 to 2.31, for example Pclass_3 from -1.70 to -0.81 and Sex from -2.64 to -1.96 . L1 sets coefficients exactly to zero: at $\lambda = 0.05$ six of nine are zero, with Sex and Fare the last to disappear along the path (Figure 2), and at the selected $\lambda = 0.003$ it already drove Embarked_Q to 0.016. Sex is by far the dominant signal (coefficient about -2.6 unregularised), followed by third class (about -1.7); age, fare, second class and sibling count each sit around 0.4–0.65 in magnitude. L1 gave the best logistic test AUC (0.844) and is the most interpretable choice, but its advantage over the other two is far inside the noise.

4. Bagging versus boosting

Bagging. The from-scratch bagger (200 bootstrap trees; depth 8 and the number of trees were chosen by out-of-bag accuracy, which rose from 0.789 at depth 1 to 0.839 at depth 8) reached AUC 0.846 and accuracy 0.816, the highest AUC of all fourteen rows in Table 1 and the best average precision among the from-scratch models. scikit-learn's random forest was equal in accuracy (0.816) and slightly lower in AUC (0.838); a plain bagged-tree ensemble with depth 4 gave 0.836. Averaging many deep, high-variance trees reduces variance without increasing bias, which suits a small, noisy problem, and the bagged ensembles and logistic models had the lowest log-loss (0.45–0.48), against 0.51–0.60 for the boosters.

Boosting. Boosting was the weakest family on the test set: AdaBoost with stumps scored AUC 0.795 from scratch and 0.826 in scikit-learn, gradient boosting 0.799 and histogram gradient boosting 0.810. Yet in cross-validation on the training data the same boosters were the strongest (CV-AUC 0.885 and 0.883, versus 0.858 for logistic regression and 0.880 for the forest). Two effects plausibly explain the reversal: sequential fitting concentrates on hard, partly random cases (survival involved luck), and the larger tuning grids make CV scores optimistic; AdaBoost's exponential loss is especially label-noise sensitive and its probabilities were poorly calibrated (log-loss 0.55 and 0.60). The from-scratch and scikit-learn AdaBoost agree on accuracy (0.799) but differ in AUC, most likely because of different ensemble sizes (200 versus 100 stumps) and probability mappings.

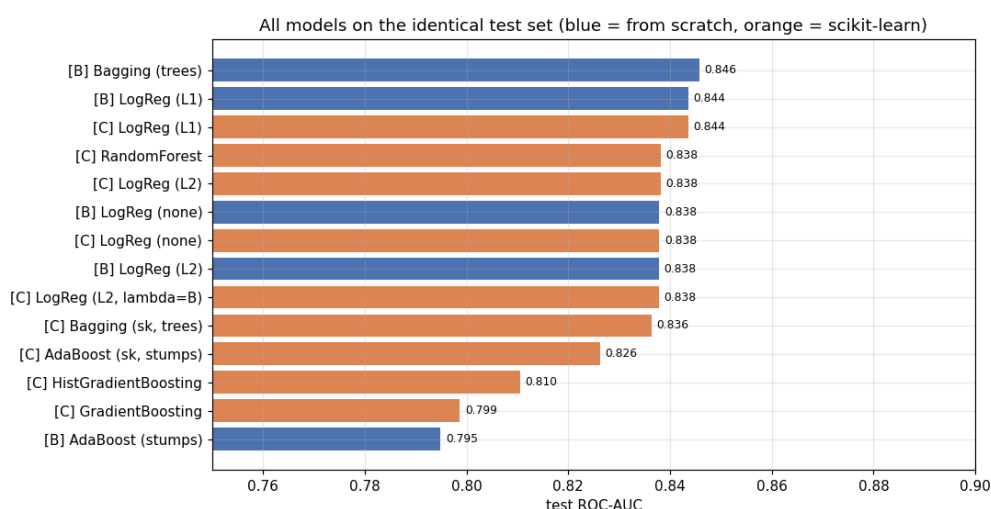


Figure 1. Test ROC-AUC of every model (blue: from scratch, orange: scikit-learn). The bootstrap intervals overlap almost entirely.

5. Performance conclusion

No approach is demonstrably best. On the test set the top models (from-scratch bagging, L1 logistic regression) lead by under 0.01 AUC and by at most one or two correctly classified passengers, well inside the intervals. Cross-validation, which uses 712 rather than 179 evaluation rows, suggests that the random forest and the two gradient boosters gain about 0.02–0.03 AUC over logistic regression, plausibly by capturing interactions such as Sex × Pclass, whereas the test split favours the linear models. For this dataset a regularised logistic regression or a bagged ensemble is the sensible choice: they are cheap, stable, and the logistic models are interpretable. Regularisation mattered for interpretation and sparsity rather than accuracy. Runtime was modest: all notebooks run in about 2.5 minutes in total on a CPU (library timings include grid search; scratch timings are final fits only).

Limitations. Results come from one split with 179 test rows, so every ranking above is tentative. Tuning budgets were not identical: the from-scratch logistic and AdaBoost models used cross-validation over small grids, the bagger used out-of-bag accuracy, and the library models used larger grids, so differences between implementations partly reflect tuning rather than code. The from-scratch learners were deliberately simple (Gini trees without pruning, discrete AdaBoost with weighted-error stumps and a logistic link for probabilities), which is why scikit-learn's gradient boosting variants, which have no counterpart in Section B, are reported separately.

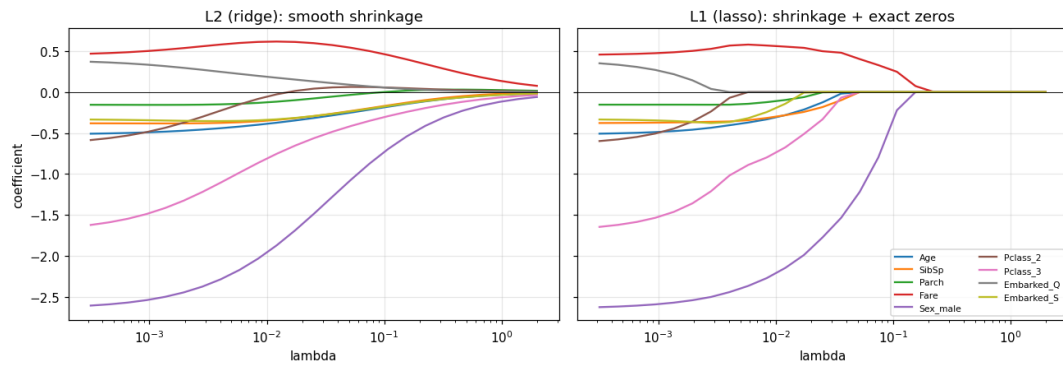


Figure 2. Coefficient paths of the from-scratch L2 (left) and L1 (right) logistic regressions.

6. Future recommendations

- **Richer features.** Extract the title from Name (Master, Miss, Mrs), family size and an is-alone flag, ticket-group size and deck from Cabin, and impute Age by title. These capture the age, sex and family effects that the minimal pipeline leaves in free text; add a Sex $\times$ Pclass interaction to logistic regression.
- **More reliable evaluation.** Replace the single 179-row test comparison with repeated stratified k-fold (or nested) CV and paired tests or paired bootstrap on per-fold differences, so that gaps of one or two points are not over-interpreted.
- **Calibration and thresholds.** Calibrate probabilities (Platt or isotonic) for the boosters and pick the decision threshold from a cost or F1 criterion on CV predictions; our sweep for the from-scratch L2 model shows that threshold 0.4 raises recall from 0.67 to 0.75 at the same F1 (0.74). Then test a stacked ensemble of logistic regression and histogram gradient boosting.